

Simple Geometry Solutions - No Cross Product

C++17 interview implementations using direct algebra, projection, ray casting, and basic coordinate formulas.

Problem	Method	Time
Line segment intersection	Parametric t/u	O(1)
Point-to-segment distance	Projection	O(1)
Point inside polygon	Ray casting	O(n)
Polygon area	Shoelace	O(n)
Circle-line intersection	Quadratic	O(1)
Coordinate transforms	Rotation + translation	O(1)
Angle wrapping	$\pm 2\pi$	O(1) typical

1. Line Segment Intersection

Method	Parametric lines	Time	O(1)
--------	------------------	------	------

Idea: Write both segments as $A + t(B-A)$ and $C + u(D-C)$. Solve for t and u . The segments intersect when both values are between 0 and 1.

Implementation: If the denominator is zero, the lines are parallel. Then check whether any endpoint lies on the other segment using the ordinary line equation $Ax + By + C = 0$ and bounding-box checks.

Remember: t tells where the intersection lies on the first segment; u does the same for the second.

2. Point-to-Segment Distance

Method	Projection parameter t	Time	O(1)
--------	--------------------------	------	------

Idea: Project the point onto the infinite line through the segment. The scalar t tells where that projection lands relative to the endpoints.

Implementation: Clamp t to $[0,1]$. Then compute the closest point $A + t(B-A)$ and take ordinary Euclidean distance.

Remember: $t < 0$ means A is closest; $t > 1$ means B is closest.

3. Point Inside Polygon

Method	Ray casting	Time	O(n)
--------	-------------	------	------

Idea: Imagine a horizontal ray starting at the query point and going to the right. Count how many polygon edges that ray crosses.

Implementation: Instead of explicitly counting, toggle a boolean every time an edge crosses the ray. Odd crossings mean inside; even crossings mean outside.

Remember: For each edge that spans the point's y-coordinate, compute the edge's x-coordinate at that same y-value.

4. Polygon Area

Method	Shoelace formula	Time	$O(n)$
--------	------------------	------	--------

Idea: Walk around the polygon in vertex order. Add $x_i y_{next}$ and subtract $y_i x_{next}$ for every edge.

Implementation: Take the absolute value of the total and divide by 2.

Remember: This works for clockwise or counterclockwise vertex order.

5. Circle-Line Intersection

Method	Substitution + quadratic formula	Time	$O(1)$
--------	----------------------------------	------	--------

Idea: Represent the line as $P(t) = A + t(B-A)$. Substitute its x and y expressions into the circle equation.

Implementation: The result is a quadratic $At^2 + Bt + C = 0$. Its discriminant determines whether there are 0, 1, or 2 intersections.

Remember: After solving for t, plug each t back into the line equation to recover the intersection point.

6. Coordinate Transforms

Method	2D rotation + translation	Time	$O(1)$
--------	---------------------------	------	--------

Idea: To convert local to world coordinates, rotate the local point by the robot heading and then add the robot position.

Implementation: To convert world to local, subtract the robot position first, then rotate by the negative heading.

Remember: Local->World: $p_w = R(\theta)p_l + t$. World->Local: $p_l = R(-\theta)(p_w - t)$.

7. Angle Wrapping

Method	Repeated $\pm 2\pi$	Time	$O(k)$, normally $O(1)$
--------	---------------------	------	--------------------------

Idea: Angles that differ by 2π represent the same direction. Repeatedly subtract 2π when the angle is too large and add 2π when it is too small.

Implementation: The implementation returns angles in $[-\pi, \pi)$.

Remember: Always wrap angle differences such as target-current before using them in control or estimation.